Chương 6: Lập trình phía máy chủ: ASP MVC

TS Nguyễn Mạnh Tuấn

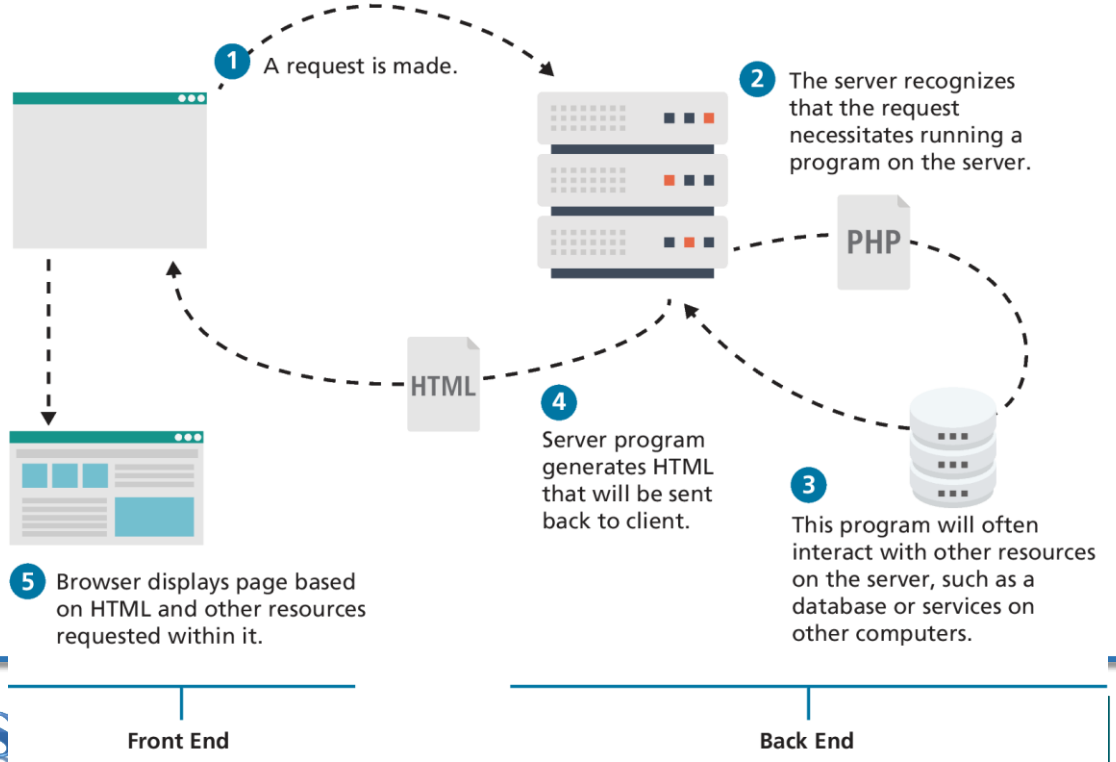
tuannm@ueh.edu.vn

Nội dung

- **Lập trình máy chủ**
- **Tổng quan ASP MVC**
  - Khái niệm, nguyên lý, lịch sử phát triển
  - MVC Frameworks
- **Dự án ASP MVC**
  - Tạo project ASP MVC Core
  - Thêm mới Controller
  - Thêm mới View
  - Thêm mới Model
  - Làm việc với cơ sở dữ liệu
  - Thêm tính năng tìm kiếm
  - Thêm thuộc tính mới
  - Tạo những ràng buộc nhập liệu

Front End vs Back End

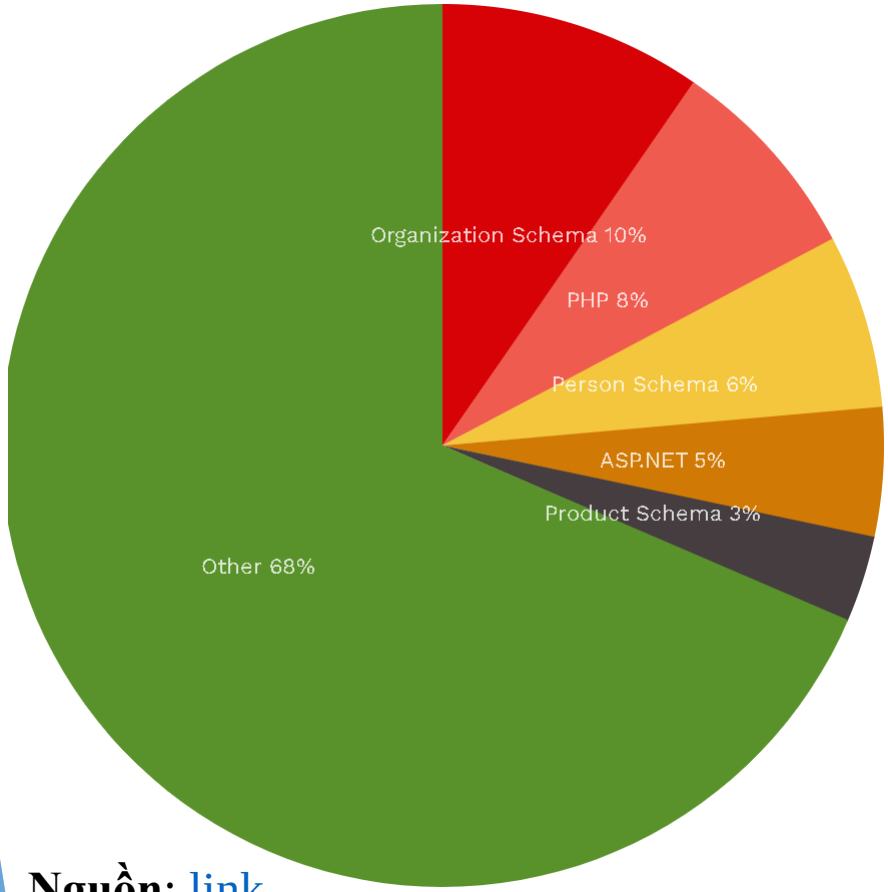
- Công nghệ phía máy chủ cung cấp quyền truy cập vào nguồn dữ liệu, xử lý bảo mật và cho phép các trang web tương tác với các dịch vụ bên ngoài như hệ thống thanh toán.



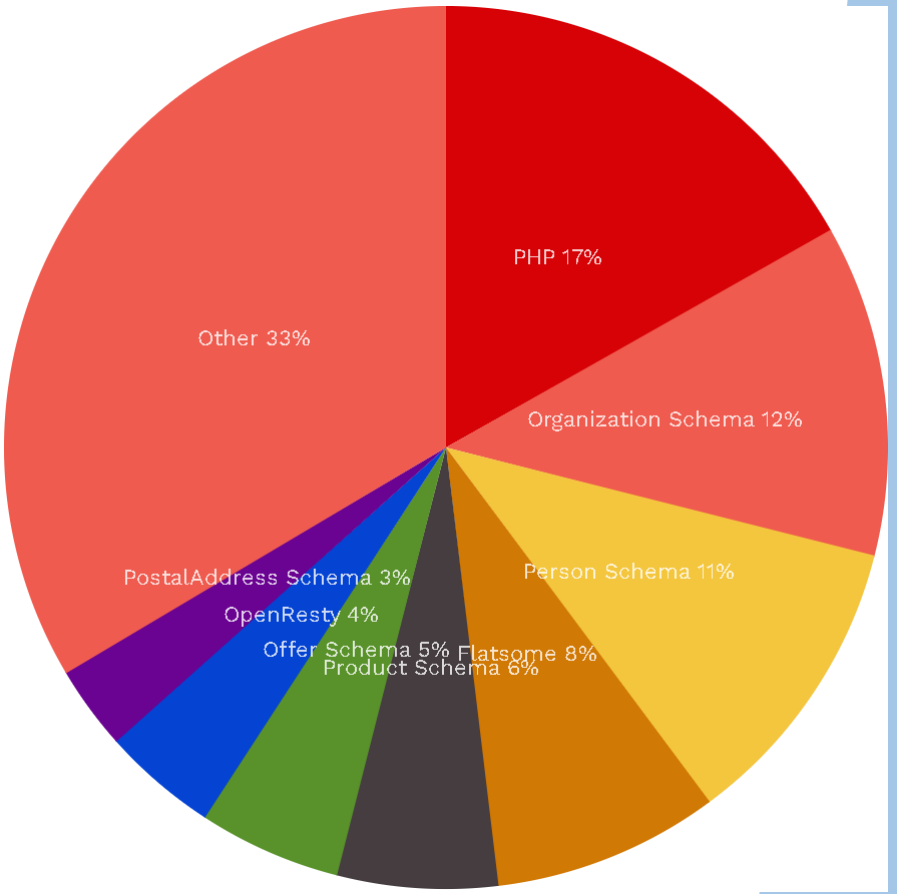
Công nghệ máy chủ phổ biến

- **ASP (Active Server Pages).** Đây là công nghệ phía máy chủ đầu tiên của Microsoft (còn gọi là ASP Classic).
- **ASP.NET.** Công nghệ này thay thế công nghệ ASP cũ của Microsoft.
- **JSP (Java Server Pages).** JSP sử dụng Java làm ngôn ngữ lập trình và giống như ASP.NET, nó sử dụng phương pháp hướng đối tượng rõ ràng và được sử dụng trong các hệ thống web doanh nghiệp lớn và được tích hợp vào môi trường J2EE.
- **Node.js (** hoặc chỉ **Node).** Sử dụng JavaScript ở phía máy chủ
- **Perl** rất giỏi trong việc xử lý văn bản.
- **PHP.** Giống như ASP, PHP là ngôn ngữ được gõ động có thể được nhúng trực tiếp vào HTML
- **Python.** Ngôn ngữ lập trình hướng đối tượng ngắn gọn này có nhiều công dụng, bao gồm cả việc dùng để tạo các ứng dụng web.
- **Ruby on Rails.** Đây là một khuôn khổ phát triển web sử dụng ngôn ngữ lập trình Ruby.

Frameworks Usage Distribution in the Top 1 Million Sites



Vietnam



Nguồn: link

Tổng quan

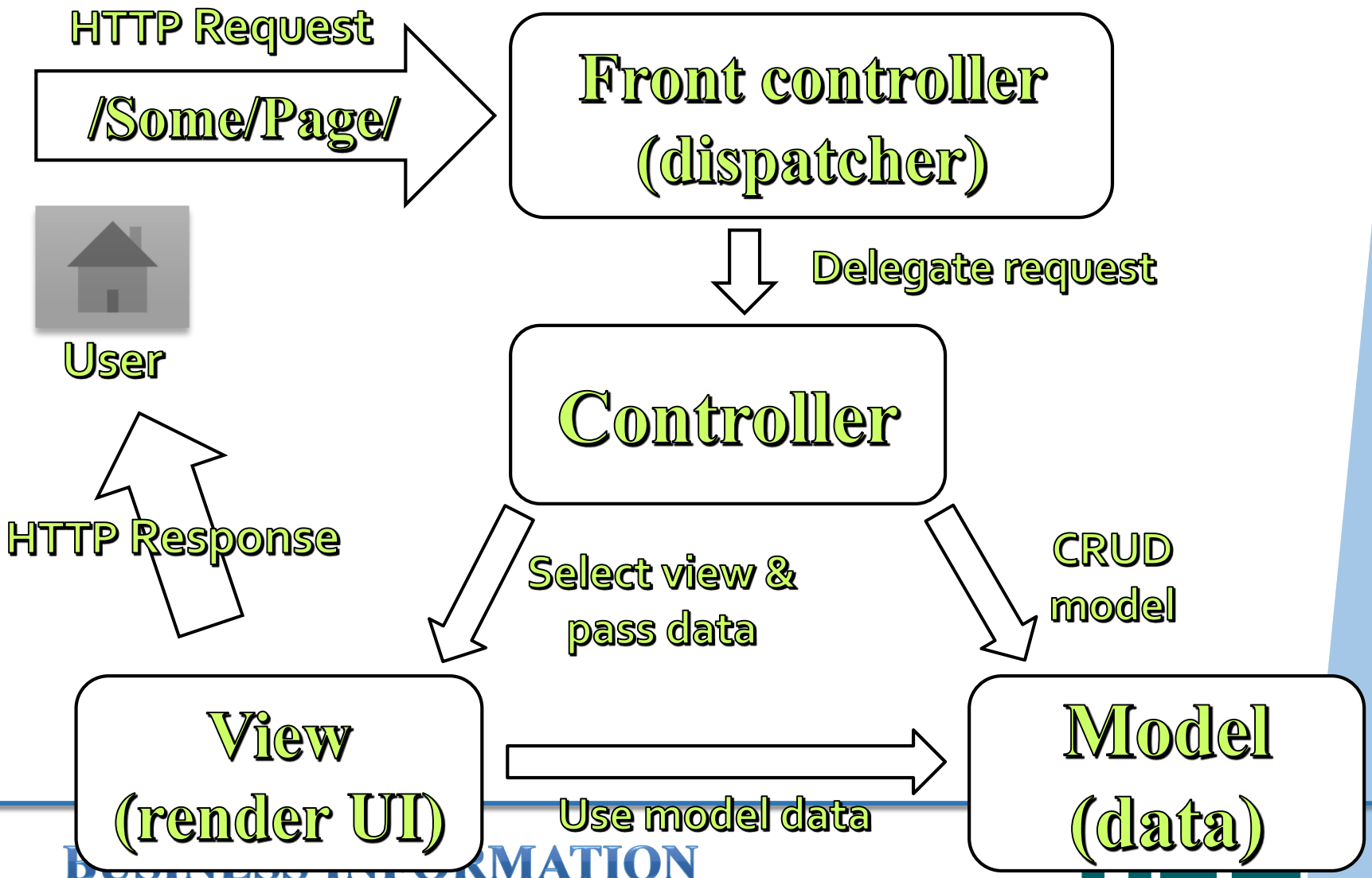
- ASP.NET MVC là một framework phát triển dùng cho việc xây dựng trang web và website với HTML5, CSS3, JavaScript (Jquery, Bootrap, Angular…) và mã kịch bản server.
- **Wikipedia: "Model–view–controller** (**MVC**) is a software architectural pattern mostly (but not exclusively) for implementing user interfaces on computers.
- Những trang Web đang sử dụng ASP.NET MVC tại VN

Tổng quan

- MVC là 1 framework dùng để xây dựng ứng dụng website dùng thiết kế MVC (Model View Controller)

- MVC chia ứng dụng web thành 3 tầng:
  - The business layer (Model logic)
  - The display layer (View logic)
  - The input control (Controller logic)

The MVC Pattern for Web



Khái niệm

- **Model** là lõi ứng dụng (thể hiện danh sách dữ liệu).
  – Model điều khiển logic cho các ứng dụng dữ liệu. Thông thường các đối tượng mô hình truy xuất dữ liệu (hoặc là lưu trữ dữ liệu) từ database
  – Model trực tiếp quản lý dữ liệu, logic và các luật của ứng dụng.

Khái niệm

- **View** là nơi điều khiển hiển thị dữ liệu.
  – Đa số View được tạo từ dữ liệu mô hình.
  – View hiển thị dữ liệu đầu ra của dữ liệu như biểu đồ hay mô hình.

Khái niệm

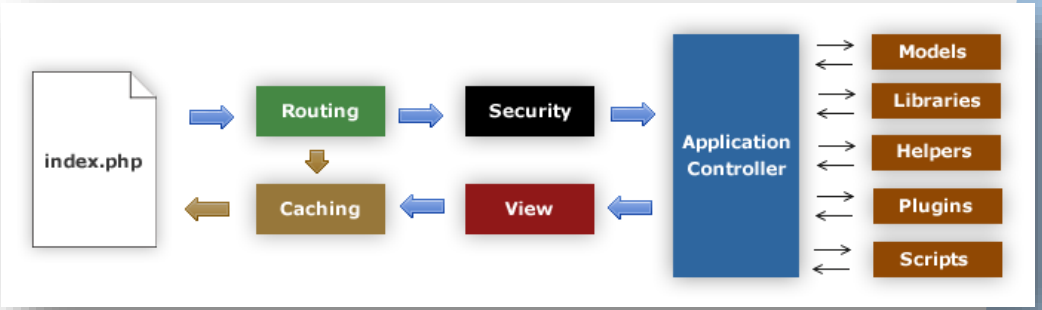
- **Controller** điều khiển dữ liệu tương tác với database.
  – Controller nhận dữ liệu đầu vào và chuyển dữ liệu này thành lệnh để Model hoặc View thực thi.
  – Controller là 1 phần của ứng dụng để điều khiển tương tác giữa user và database.
  – Thông thường Controller đọc dữ liệu từ View, Input và gửi dữ liệu đến Model

Ưu điêm và nhược điểm của mô hình MVC

- *1. Ưu điểm:*
  - Dễ dàng lập trình, nâng cấp, debug và test ứng dụng nhờ được chia thành các thành phần độc lập,
  - Thích hợp cho các dự án lớn, gồm nhiều người giúp dễ dàng làm việc và phân chia công việc với các thành viên
- *2. Nhược điểm:*
  - Không thích hợp cho dự án nhỏ do bạn sẽ phải tốn nhiều thời gian cho quá trình phát triển theo mô hình MVC.

MVC Frameworks

- CakePHP (PHP)
- CodeIgniter (PHP)
- Spring (Java)
- Perl: Catalyst, Dancer
- Python: Django, Flask, Grok
- Ruby: Ruby on Rails, Camping, Nitro, Sinatra
- JavaScript: AngularJS, JavaScriptMVC, Spine
- ASP.NET MVC (.NET Framework)



The ASP.NET MVC History

- ASP.NET MVC 1.0
  - In February 2007, Scott Guthrie ("ScottGu") of Microsoft sketched out the core of ASP.NET MVC
  - Released on 13 March 2009
- ASP.NET MVC 2.0 (Areas, Async)
  - Released just one year later, on 10 March 2010
- ASP.NET MVC 3.0 (Razor) – 13 January 2011
- ASP.NET MVC 4.0 (Web API) – 15 August 2012
- ASP.NET MVC 5.0 (Identity) – 17 October 2013
- ASP.NET MVC 5.2.3 – 9 February 2015
- ASP.NET Core 1.0 – 2016

Separation of Concerns

- Each component has one responsibility
  - SRP – Single Responsibility Principle
  - DRY – Don't Repeat Yourself
- More easily testable
  - TDD – Test-driven development
- Helps with concurrent development
  - Performing tasks concurrently
    - One developer works on views
    - Another works on controllers

Extensible

- Replace any component of the system
  - Interface-based architecture
- Almost anything can be replaced or extended
  - Model binders (request data to CLR objects)
  - Action/result filters (e.g. OnActionExecuting)
  - Custom action result types
  - View engine (Razor, WebForms, NHaml, Spark)
  - View helpers (HTML, AJAX, URL, etc.)
  - Custom data providers (ADO.NET), etc.
  - Controller factory can be also replaced

Clean URLs

- REST-like
  - /products/update
  - /blog/posts/2013/01/28/mvc-is-cool
- Friendlier to humans
  - /product.aspx?catId=123 or post.php?id=123
  - Becomes /products/chocolate/
- Friendlier to web crawlers
  - Search engine optimization (SEO)

Community-based

- ASP.NET MVC, Web API, and Web Pages source code is available in CodePlex
  - http://aspnetwebstack.codeplex.com/
- You can vote for new features in ASP.NET UserVoice site
  - http://aspnet.uservoice.com/forums/41201-asp-net-mvc
- ASP.NET Core (the new MVC) is on GitHub
  - https://github.com/aspnet
  - https://github.com/aspnet/Mvc

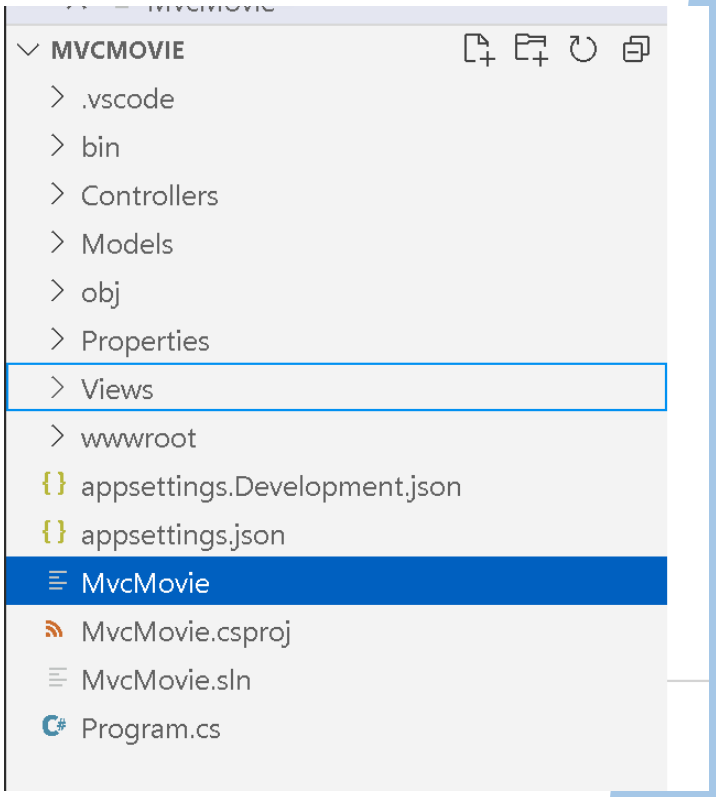
Creating ASP.NET MVC Project

Tạo project

- Mở VS Code và thực hiện các bước sau:
  - Chọn **New Terminal** từ menu **Terminal**
  - Chuyển đến thư mục (cd) sẽ chứa dự án
  - Chạy các lệnh sau:
    - **dotnet new mvc -o MvcMovie**
    - **code -r MvcMovie**
  - **Chọn View > Command Palette**
    - Select the .NET: Generate Assets for Build and Debug

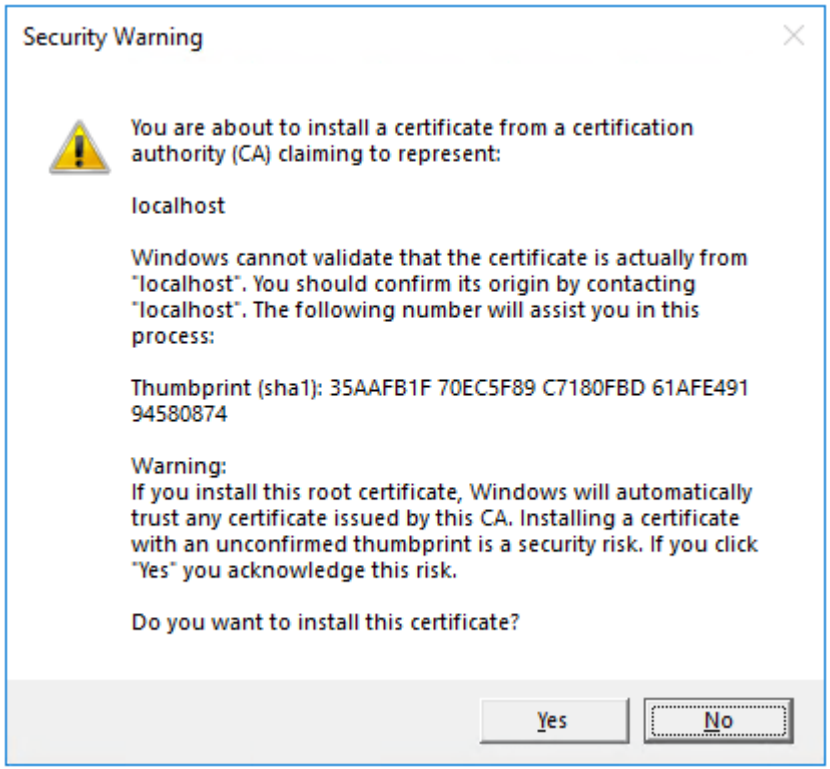
Tạo project

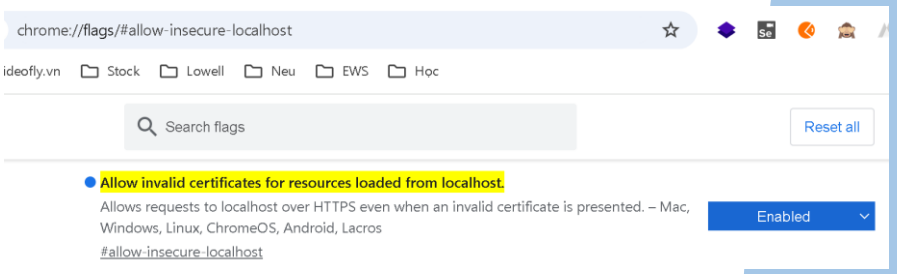
- Lệnh dotnet new mvc
  - tạo một dự án ASP.NET Core MVC mới trong thư mục MvcMovie
- Lệnh code -r
  - mở thư mục dự án MvcMovie trong Visual Studio Code



Chạy ứng dụng

- **Trust the HTTPS development certificate**
  - dotnet dev-certs https –trust
  - Chọn **Yes**
- **Nhấn F5 để chạy debug**
- **Trường hợp Chrome bị lỗi vì certificates**
  - chrome://flags/#allow-insecure-localhost



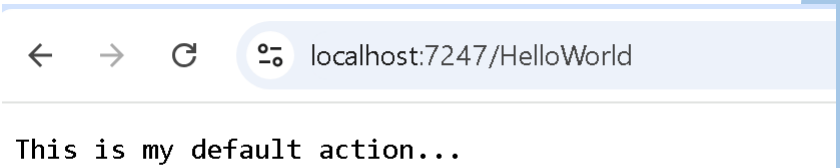


Thêm mới controller

Thêm mới controller

- Controllers > New File và đặt tên cho tệp mới HelloWorldController.cs
- **Truy cập URL:**
  - https://localhost:7247/HelloWorld

```
using Microsoft.AspNetCore.Mvc;
using System.Text.Encodings.Web;

namespace MvcMovie.Controllers
{
    public class HelloWorldController : Controller
    {
        //
        // GET: /HelloWorld/

        public string Index()
        {
            return "This is my default action...";
        }

        //
        // GET: /HelloWorld/Welcome/

        public string Welcome()
        {
            return "This is the Welcome action
method...";
        }
    }
}
```



This is my default action...

Thêm mới controller

- Giải thích cách chạy của đường dẫn
  • http://localhost:1234/HelloWorld
  ASP.NET MVC gọi các lớp controller khác nhau tùy theo URL. URL mặc định dùng bởi ASP.NET MVC như sau:
  - **[Controller]/[ActionName]/[Parameters]**
- Định dạng định tuyến được thiết lập trong Program.cs

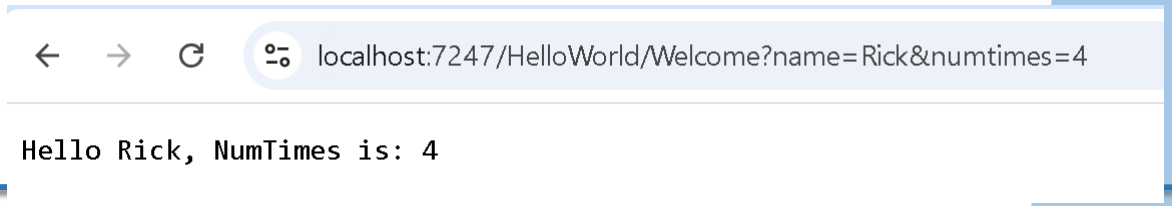
```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Thêm mới controller

- Truy cập URL:
  - https://localhost:7247/HelloWorld/Welcome
- Sửa code hàm Welcome để nhận 2 tham số

```
public string Welcome(string name, int numTimes = 1)
        {
            return HtmlEncoder.Default.Encode($"Hello
{name}, NumTimes is: {numTimes}");
        }
```

- Truy cập URL:
  - https://localhost:7247/HelloWorld/Welcome?name=Rick&numtimes=4



Thêm mới controller

- Dấu **?** chấm hỏi trong URL ở trên là dấu phân cách và theo sau là chuỗi truy vấn.
- Ký tự này **&** phân tách các cặp trường-giá trị.
- **HtmlEncoder.Default.Encode** để bảo vệ ứng dụng khỏi các dữ liệu độc hại thông qua JavaScript.

Thêm mới View

Thêm mới View

- Trong phần này sẽ thêm các View (giao diện) cho ứng dụng MVC.
- Các **View** được thể hiện bằng **cú pháp Razor** với tập tin có đuôi **.cshtml**.
- Razor giảm thiểu số lượng ký tự và từ khóa để hiển thị HTML khi lập trình.

- Tìm hiểu về Razor: http://www.w3schools.com/aspnet/razor_intro.asp

Thêm mới View

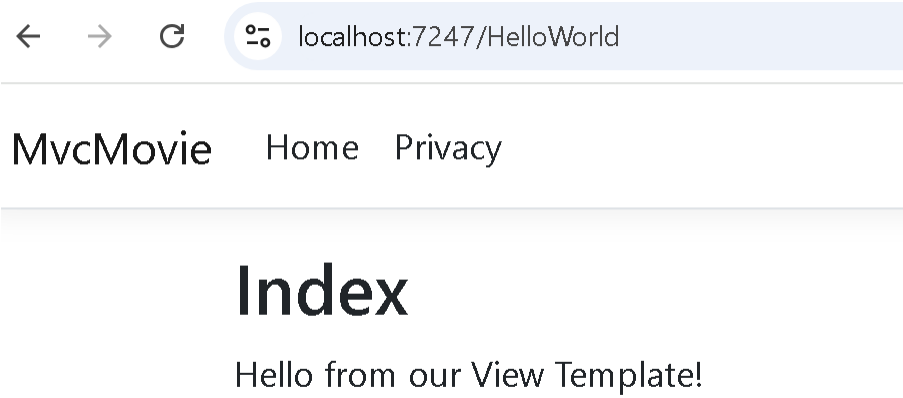
- Thêm một thư mục mới có tên là *Views/HelloWorld*
- Thêm một tệp mới vào thư mục *Views/HelloWorld* và đặt tên là *Index.cshtml*
- Thay đổi code HelloWorldController.cs

```
@{
    ViewBag.Title = "Đại học UEH";
}

<h2>Index</h2>

<p>Hello from our View Template!
@ViewBag.Title</p>
```

```
public IActionResult Index()
        {
            return View();
        }
```

Thêm mới View

- Xem kết quả tại URL: https://localhost:7247/HelloWorld

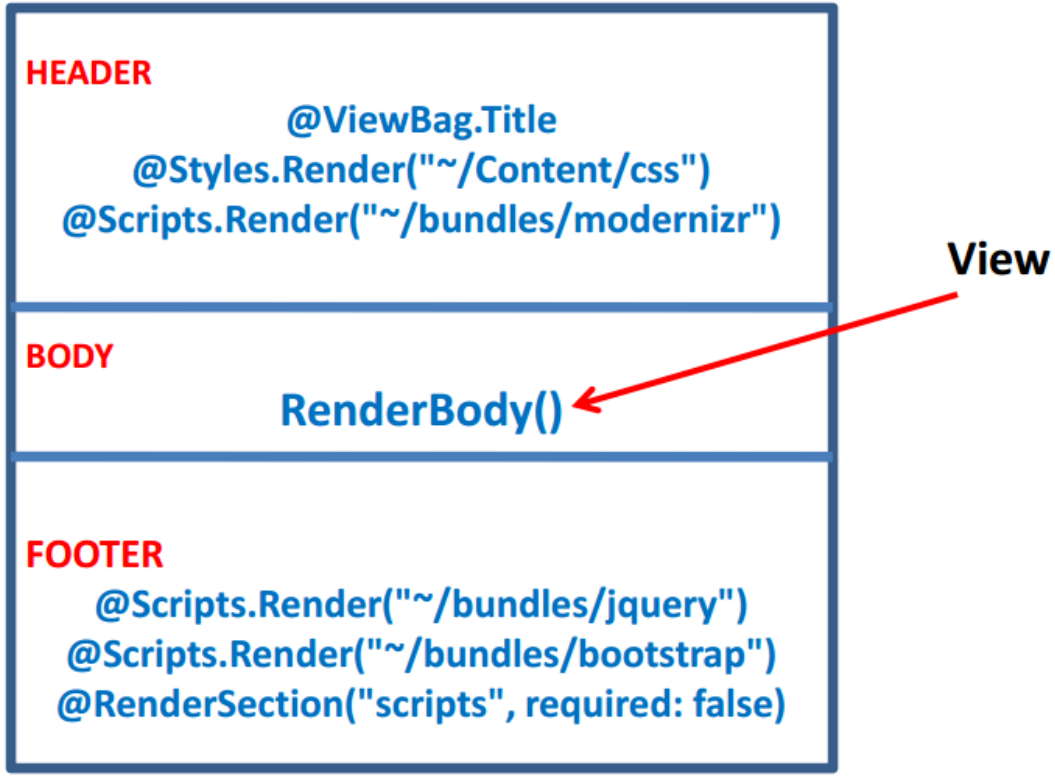


Thêm mới View

- Giải thích đoạn mã trên:
  - @{…} là cú pháp Razor
  - Layout = "~/Views/Shared/_Layout.cshtml"; đặt mặc định layout của trang là _Layout.cshtml
  - ViewBag.Title = "Index"; là thiết lập tiêu đề trang
    - ViewBag là 1 thuộc tính cho phép lấy các giá trị động từ Controller lên View.
    - ViewBag là đối tượng động và không có kiểu định nghia trước.

Thay đổi View và các trang Layout

- Tất cả các trang View thông thường sử dụng 1 Layout duy nhất đó là ***_Layout.cshtml*** ở thư mục ***/Views/Shared.***
- Thay đổi nội dung **_Layout.cshtml** tức là thay đổi bố cục cho tất cả trang View dùng trang này làm trang Layout



Thay đổi tiêu đề, chân trang và liên kết menu trong tệp bố cục

- Thay thế nội dung của Views/Shared/_Layout.cshtml
  - <title>@ViewData["Title"] - MvcMovie</title>
    - <title>@ViewData["Title"] - Movie App</title>
  - <a class="navbar-brand" asp-area="" asp-controller="Home" asp-action="Index">MvcMovie</a>
    - <a class="navbar-brand" asp-area="" asp-controller="Movies" asp-action="Index">Movie App</a>
  - © 2024 - MvcMovie - <a asp-area="" asp-controller="Home" asp-action="Privacy">Privacy</a>
    - © 2024 - Movie App - <a asp-area="" asp-controller="Home" asp-action="Privacy">Privacy</a>

Truyền dữ liệu từ Controller đến View

- Chỉnh hàm Welcome trong HelloWorldController.cs

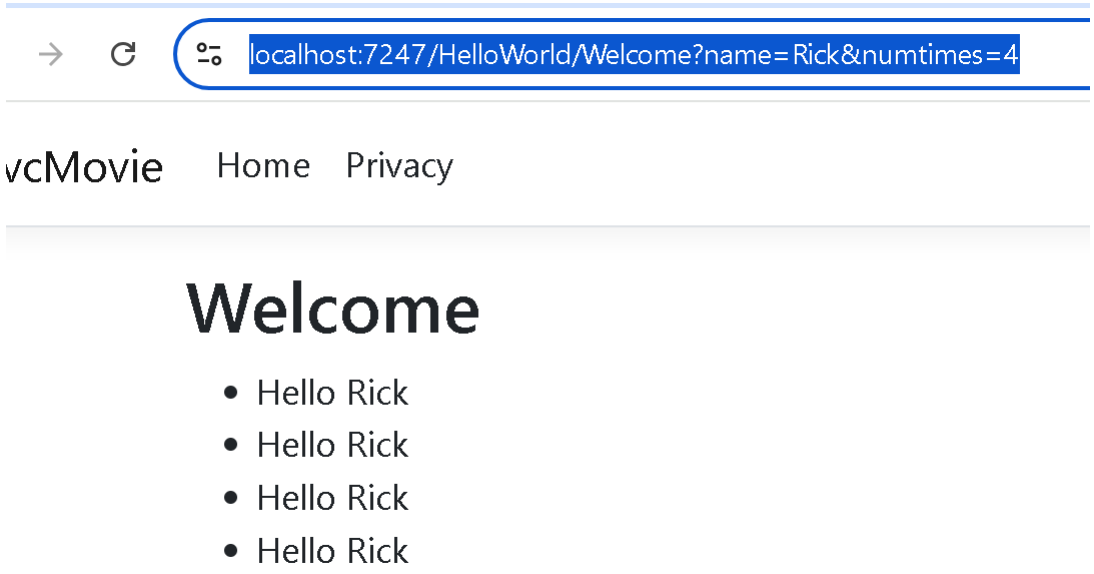
```
public IActionResult Welcome(string name, int numTimes = 1)
        {
            ViewData["Message"] = "Hello " + name;
            ViewData["NumTimes"] = numTimes;

            return View();
        }
```

- Tạo mới Views/HelloWorld/Welcome.cshtml

```
@{
    ViewData["Title"] = "Welcome";
}

<h2>Welcome</h2>

<ul>
    @for (int i = 0; i < (int)ViewData["NumTimes"]!; i++)
    {
        <li>@ViewData["Message"]</li>
    }
</ul>
```

Truyền dữ liệu từ Controller đến View

- Chạy URL
  - https://localhost:7247/HelloWorld/Welcome?name=Rick&numtimes=4



Chuyển dữ liệu từ Controller sang View

- Trong ví dụ trên, biến **ViewData** được dùng để đẩy dữ liệu từ Controller sang View.
- Ngoài ra, người ta còn dùng **View Model** để đẩy dữ liệu từ Controller sang View. Cách này được dùng thông dụng và ưa chuộng hơn ViewBag.

- **Xem thêm:** https://learn.microsoft.com/en-us/aspnet/core/mvc/views/overview?view=aspnetcore-8.0

Thêm mới Model

Thêm mới Model

- Phần này sẽ thêm các lớp để quản lý các bảng dữ liệu ở database.
- Thông thường dùng **Entity Framework** để định nghia mô hình và các lớp mã nguồn.
- EF cũng support Code First, Database First hay Model First ở phần Model.
- Có thể nhúng dự án thư viện mô hình DLL ở bên ngoài thay thế cho phần Model.

Thêm mới Model

- **Tạo mới file** Models/Movie.cs

```
using System.ComponentModel.DataAnnotations;

namespace MvcMovie.Models
{
    public class Movie
    {
        public int Id { get; set; }
        public string? Title { get; set; }

        [DataType(DataType.Date)]
        public DateTime ReleaseDate { get; set; }
        public string? Genre { get; set; }
        public decimal Price { get; set; }
    }
}
```

Thêm các gói NuGet

- Từ Terminal chạy các lệnh sau:
  - dotnet tool uninstall --global dotnet-aspnet-codegenerator
  - dotnet tool install --global dotnet-aspnet-codegenerator
  - dotnet tool uninstall --global dotnet-ef
  - dotnet tool install --global dotnet-ef
  - dotnet add package Microsoft.EntityFrameworkCore.Design
  - dotnet add package Microsoft.EntityFrameworkCore.SQLite
  - dotnet add package Microsoft.VisualStudio.Web.CodeGeneration.Design
  - dotnet add package Microsoft.EntityFrameworkCore.SqlServer
  - dotnet add package Microsoft.EntityFrameworkCore.Tools

Scaffold movie pages

- Từ Terminal chạy các lệnh sau:
  - dotnet aspnet-codegenerator controller -name MoviesController -m Movie -dc MvcMovie.Data.MvcMovieContext --relativeFolderPath Controllers --useDefaultLayout --referenceScriptLibraries --databaseProvider sqlite

Parameter	Description
-m	The name of the model.
-dc	The data context.
--relativeFolderPath	The relative output folder path to create the files.
--useDefaultLayout|-udl	The default layout should be used for the views.
--referenceScriptLibraries	Adds `_ValidationScriptsPartial` to Edit and Create pages.
-sqlite	Flag to specify if `DbContext` should use SQLite instead of SQL Server.

```
Generating a new DbContext class 'MvcMovie.Data.MvcMov
Attempting to compile the application in memory with t
Attempting to figure out the EntityFramework metadata

Using database provider 'Microsoft.EntityFrameworkCore

Added DbContext : '\Data\MvcMovieContext.cs'
Added Controller : '\Controllers\MoviesController.cs'.
Added View : \Views\Movies\Create.cshtml
Added View : \Views\Movies\Edit.cshtml
Added View : \Views\Movies\Details.cshtml
Added View : \Views\Movies\Delete.cshtml
Added View : \Views\Movies\Index.cshtml
RunTime 00:00:18.63
```

Use SQLite for development, SQL Server for production

- Thêm code sau vào Program.cs

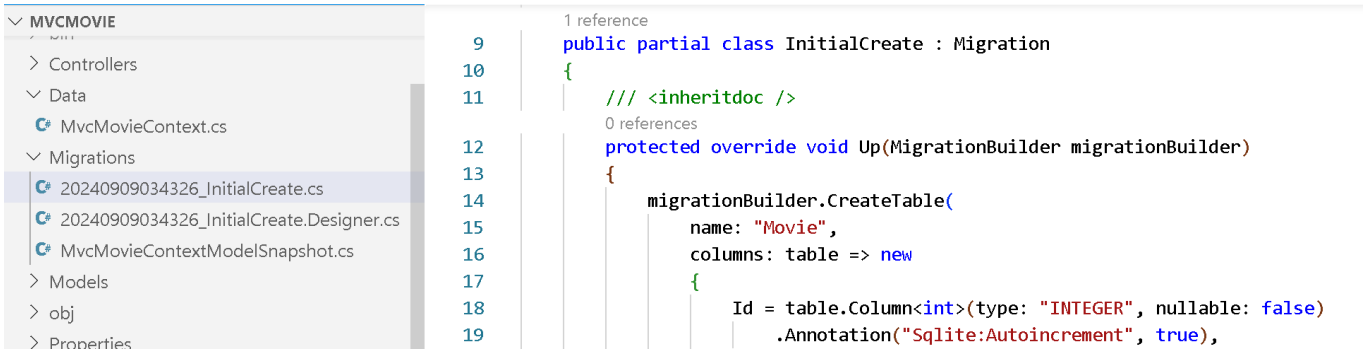
```
var builder = WebApplication.CreateBuilder(args);

if (builder.Environment.IsDevelopment())
{
    builder.Services.AddDbContext<MvcMovieContext>(options =>
        options.UseSqlite(builder.Configuration.GetConnectionStrin
g("MvcMovieContext")));
}
else
{
    builder.Services.AddDbContext<MvcMovieContext>(options =>
        options.UseSqlServer(builder.Configuration.GetConnectionSt
ring("ProductionMvcMovieContext")));
}
```

Initial migration

- Từ Terminal chạy các lệnh sau:
  - dotnet ef migrations add InitialCreate
    - Kiểm tra những thay đổi từ code và DB
  - dotnet ef database update
    - Cập nhật thay đổi lên DB

Initial migration

- dotnet ef migrations add InitialCreate

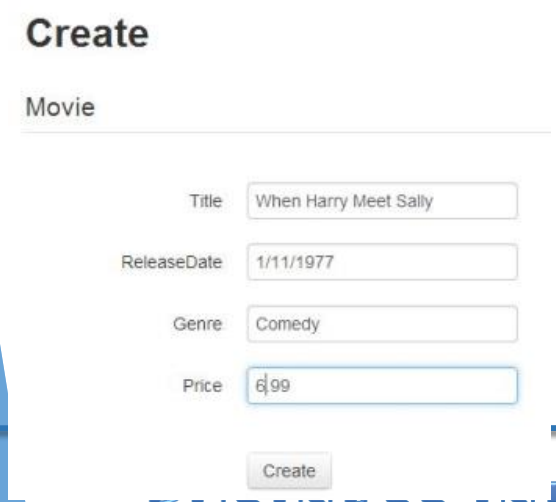


- dotnet ef database update

```
PS D:\Teaching\LuuTru\Teaching2\PTUDTMDT\Exercises\Back_End\MvcMovie> dotnet ef database update
Build started...
Build succeeded.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (15ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      PRAGMA journal_mode = 'wal';
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (5ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      CREATE TABLE "__EFMigrationsHistory" (
          "MigrationId" TEXT NOT NULL CONSTRAINT "PK___EFMigrationsHistory" PRIMARY KEY,
          "ProductVersion" TEXT NOT NULL
```

Tạo dữ liệu Movies

- Chạy chương trình và truy cập URL:
  - https://localhost:7247/Movies
- Chọn **Create New** thì giao diện tạo mới hiển thị với đường dẫn ***http://localhost:xxxxx/Movies/Create***



Hiểu code

- Giải thích phương thức **Details trong MoviesController.cs:**
  - **int?**
  - http://localhost:1234/movies/details/1
  - http://localhost:1234/movies/details?id=1
  - **return View(movie);**

Hiểu code

- Giải thích ***Views\Movies\Details.cshtml***:
  - @model MvcMovie.Models.Movie
  - @Html.DisplayNameFor(model => model.Title)
  - <a asp-action="Edit" asp-route-id="@Model?.Id">Edit</a>
  - <a asp-action="Index">Back to List</a>

Hiểu code

- Phương thức **Index** ở *MoviesController.cs*

```
public async Task<IActionResult> Index()
        {
            return View(await
_context.Movie.ToListAsync());
        }
```

Ở ***Index.cshtml*** chứa chỉ thị ở đầu file:

@model IEnumerable<MvcMovie.Models.Movie>
@model cho phép tạo ra danh sách đối tượng Movie, từ đó có thể dùng vòng lặp **foreach** để hiển thị dữ liệu trên View.

Tìm hiểu các phương thức Edit và Edit View

- Mở tập tin ***Models\Movie.cs***, chú ý các chỗ:

using System.ComponentModel.DataAnnotations;

…

```
[Display(Name = "Release Date")]
[DataType(DataType.Date)]
public DateTime ReleaseDate { get; set; }
public string? Genre { get; set; }
[Column(TypeName = "decimal(18, 2)")]
public decimal Price { get; set; }
```

Tìm hiểu các phương thức Edit và Edit View

- Giải thích các định nghĩa hiển thị cho thuộc tính public **ReleaseDate**
  - [Display(Name = "Release Date")]
  - [DataType(DataType.Date)]
  - [Column(TypeName = "decimal(18, 2)")]

Tìm hiểu các phương thức Edit và Edit View

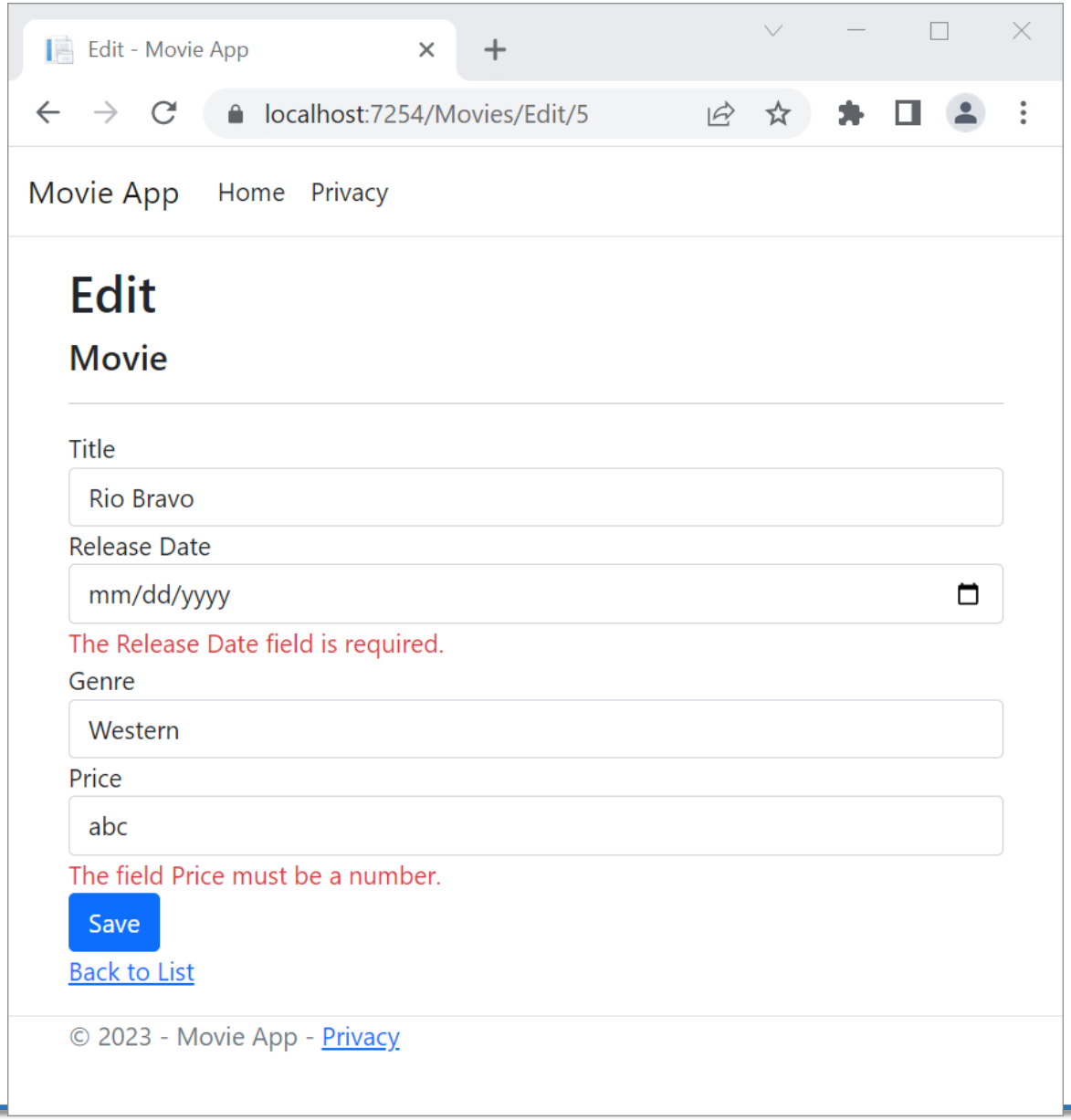
- Ở **Movie** Controller, có 2 phương thứ **Edit** như sau:
- Giải thích 2 phương thức Edit:
  - **[HttpPost]**
  - Mặc định là **[HttpGet]**
  - **[ValidateAntiForgeryToken]**: Dùng chặn giả mạo từ một Request ở Client.
  - Xem thêm: http://go.microsoft.com/fwlink/?LinkId=317598 , phần OverPosting

Tìm hiểu các phương thức Edit và Edit View

- Chạy URL *http://localhost:1234/Movies/Edit?ID=3*
- Nếu ID=3 tìm thấy trong database thì dữ liệu trả về trên giao diện ở ***Views\Movies\Edit.cshtml*** với 1 phần đoạn mã hiển thị:

```
<div class="form-group">
    <label asp-for="Title" class="control-label"></label>
    <input asp-for="Title" class="form-control" />
    <span asp-validation-for="Title" class="text-
danger"></span>
</div>
```

Kết quả


Edit - Movie App
localhost:7254/Movies/Edit/5
Movie App
Home
Privacy
Edit
Movie
Title
Rio Bravo
Release Date
mm/dd/yyyy
The Release Date field is required.
Genre
Western
Price
abc
The field Price must be a number.
Save
Back to List
© 2023 - Movie App - Privacy

Làm việc với cơ sở dữ liệu

Connection String

- Trong file Program.cs:

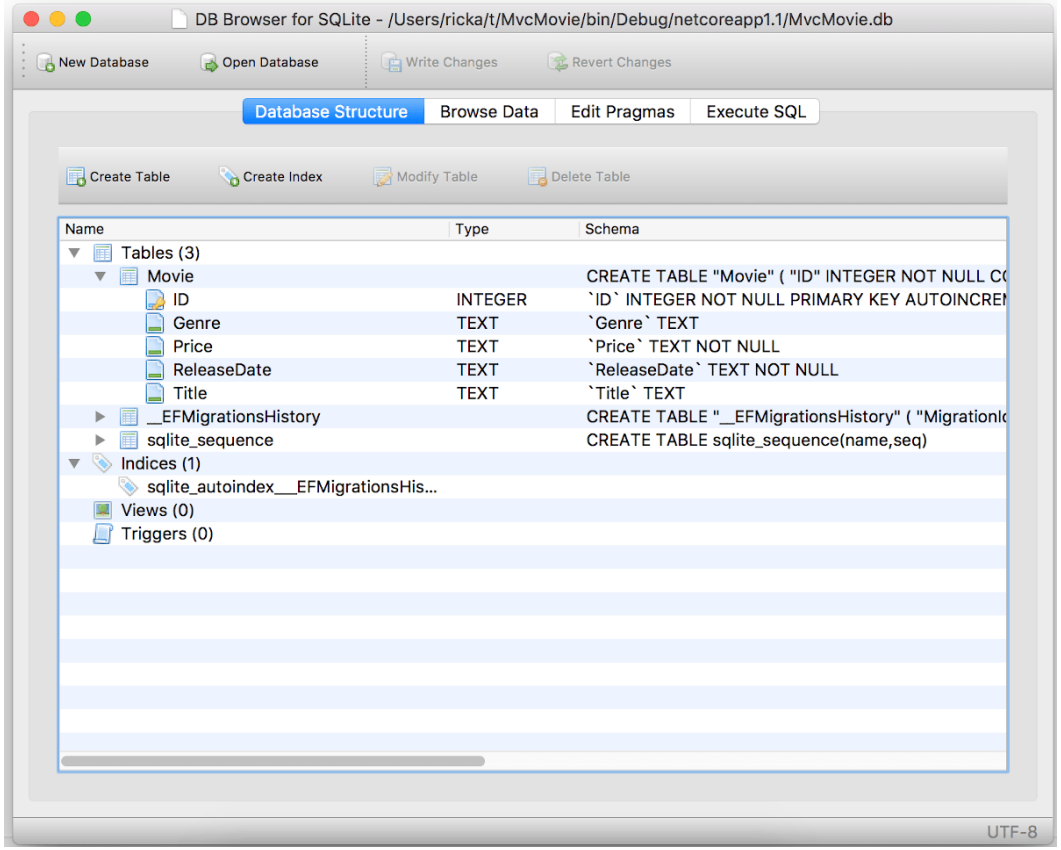
```
builder.Services.AddDbContext<MvcMovieContext>(options =>
        options.UseSqlite(builder.Configuration.GetConnectionSt
ring("MvcMovieContext")));
```

- Trong file appsettings.json:

```
"ConnectionStrings": {
  "MvcMovieContext": "Data Source=MvcMovieContext-73dbaf46-29ab-408d-9202-d87450bde5e8.db"
}
```

Truy cập SQLite

- Tải và cài đặt **DB Browser for SQLite**



Seed the database

- Tạo file SeedData.cs trong thư mục *Models*

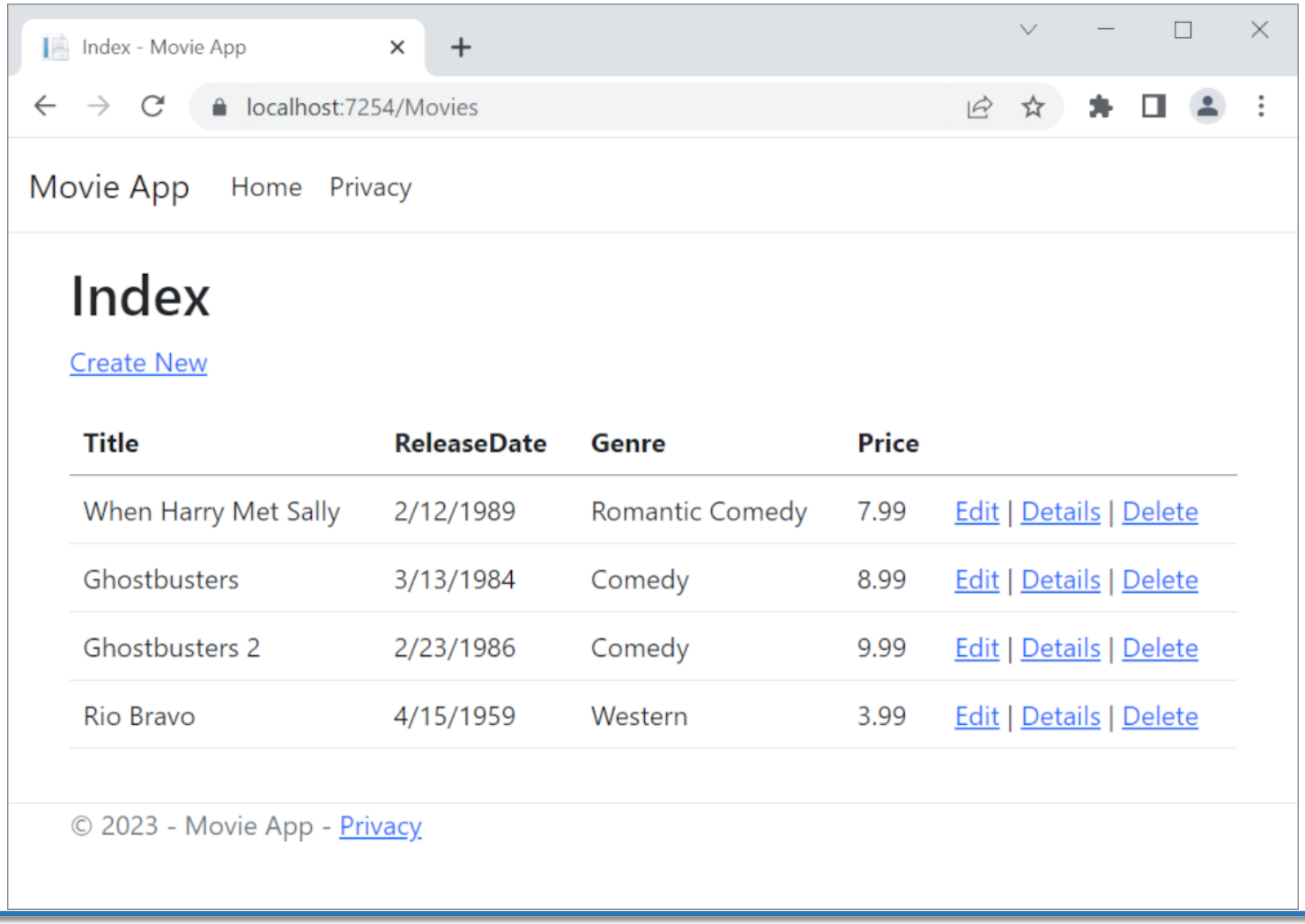
```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using MvcMovie.Data;
using System;
using System.Linq;

namespace MvcMovie.Models;

public static class SeedData
{
    public static void Initialize(IServiceProvider serviceProvider)
    {
        using (var context = new MvcMovieContext(
            serviceProvider.GetRequiredService<
                DbContextOptions<MvcMovieContext>>()))
        {
            // Look for any movies.
            if (context.Movie.Any())
            {
                return;   // DB has been seeded
            }
            context.Movie.AddRange(
                new Movie
                {
                    Title = "When Harry Met Sally",
                    ReleaseDate = DateTime.Parse("1989-2-12"),
                    Genre = "Romantic Comedy",
                    Price = 7.99M
                },
                new Movie
                {
                    Title = "Ghostbusters ",
                    ReleaseDate = DateTime.Parse("1984-3-13"),
                    Genre = "Comedy",
                    Price = 8.99M
                },
                new Movie
                {
                    Title = "Ghostbusters 2",
                    ReleaseDate = DateTime.Parse("1986-2-23"),
                    Genre = "Comedy",
                    Price = 9.99M
                },
                new Movie
                {
                    Title = "Rio Bravo",
                    ReleaseDate = DateTime.Parse("1959-4-15"),
                    Genre = "Western",
                    Price = 3.99M
                }
            );
            context.SaveChanges();
        }
    }
}
```

Seed the database

- Chỉnh code trong Program.cs

```
var app = builder.Build();

using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;

    SeedData.Initialize(services);
}
```

Kết quả


Index - Movie App
localhost:7254/Movies
Movie App
Home
Privacy
Index
Create New
Title
ReleaseDate
Genre
Price
When Harry Met Sally
2/12/1989
Romantic Comedy
7.99
Edit | Details | Delete
Ghostbusters
3/13/1984
Comedy
8.99
Edit | Details | Delete
Ghostbusters 2
2/23/1986
Comedy
9.99
Edit | Details | Delete
Rio Bravo
4/15/1959
Western
3.99
Edit | Details | Delete
© 2023 - Movie App - Privacy

Thêm tính năng tìm kiếm

Yêu cầu

- Cho phép bạn tìm kiếm phim theo *thể loại* hoặc *tên*
- Chỉnh sửa method Index trong Controllers/MoviesController.cs

```
public async Task<IActionResult> Index(string searchString)
        {
            var movies = from m in _context.Movie
                         select m;

            if (!String.IsNullOrEmpty(searchString))
            {
                movies = movies.Where(s => s.Title!.Contains(searchString));
            }

            return View(await movies.ToListAsync());
        }
```

Thử kết quả

- Tìm kiếm phim có tựa đề là **Ghost**
- Truy cập URL: https://localhost:7247/Movies/Index**?searchString=Ghost**

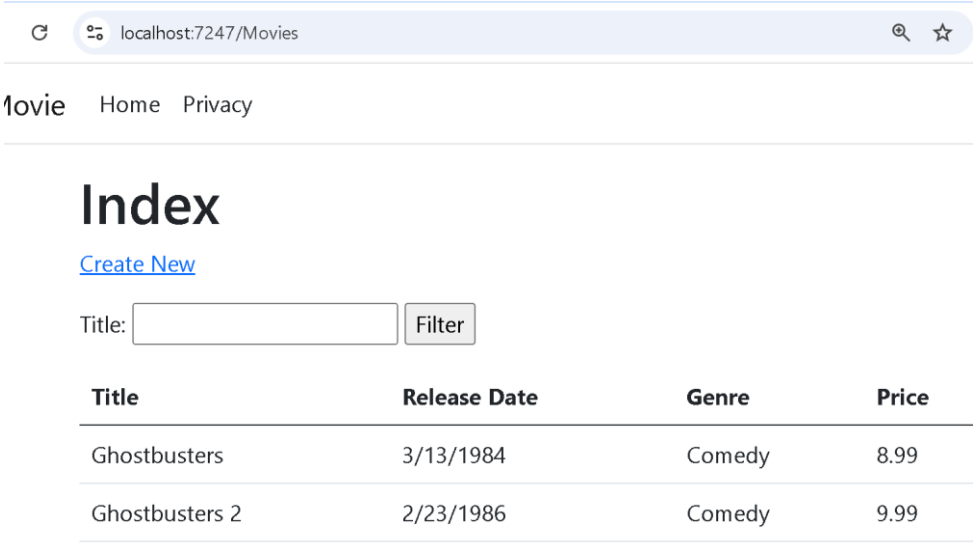
Nâng cấp

- Không thể yêu cầu người dùng sửa đổi URL mỗi lần họ muốn tìm kiếm phim.
- Mở file Views/Movies/Index.cshtml và thêm <form> sau:

```
<p>
    <a asp-action="Create">Create New</a>
</p>
<form asp-controller="Movies" asp-action="Index">
    <p>
        Title: <input type="text" name="SearchString" />
        <input type="submit" value="Filter" />
    </p>
</form>
```

Kết quả

- Truy cập URL:
  - https://localhost:7247/Movies
- Gõ chữ Ghost và tiến hành tìm kiếm

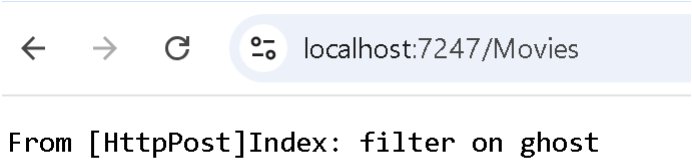


Chỉnh code

- Hãy thêm code sau vào Controllers/MoviesController.cs:

```
[HttpPost]
public string Index(string searchString, bool notUsed)
{
  return "From [HttpPost]Index: filter on " + searchString;
}
```

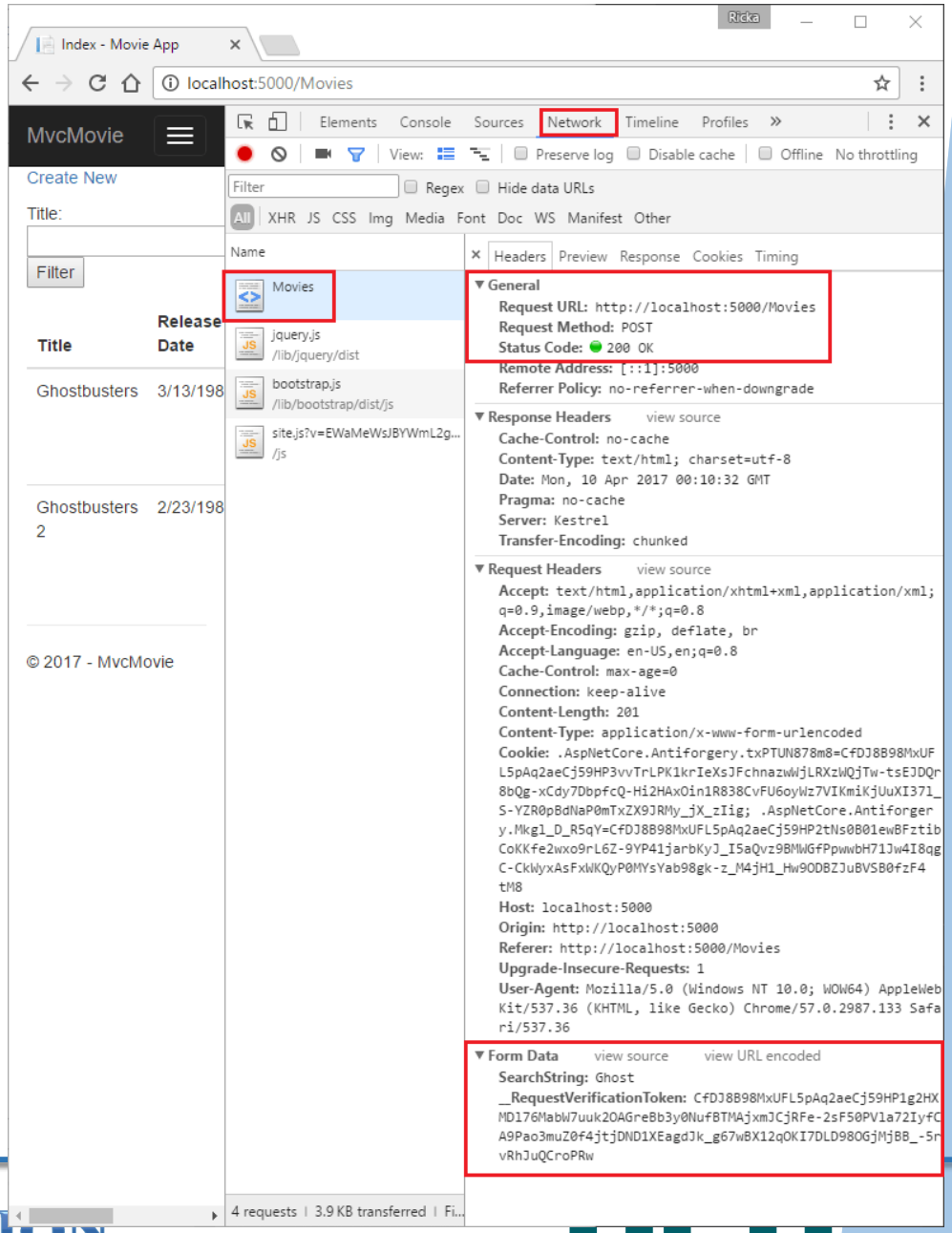
- F5 để chạy lại và thực hiện tìm kiếm lại



Giải thích

- Vậy làm sao chia sẽ liên kết tìm kiếm cho người khác

```
<form asp-controller="Movies"
asp-action="Index"
method="get">
```



Làm sao để thay đổi view không cần compile

- Cần đặt thêm **NuGet Package**
  - dotnet add package Microsoft.AspNetCore.Mvc.Razor.RuntimeCompilation
- Chỉnh code trong Program.cs
  - builder.Services.AddControllersWithViews().**AddRazorRuntimeCompilation();**

Thêm Tìm kiếm theo thể loại

- Thêm lớp MovieGenreViewModel vào thư mục Models :

```
using Microsoft.AspNetCore.Mvc.Rendering;
using System.Collections.Generic;

namespace MvcMovie.Models
{
    public class MovieGenreViewModel
    {
        public List<Movie>? Movies { get; set; }
        public SelectList? Genres { get; set; }
        public string? MovieGenre { get; set; }
        public string? SearchString { get; set; }
    }
}
```

Thêm Tìm kiếm theo thể loại

- Thay thế Index method trong MoviesController.cs bằng code sau:

```
public async Task<IActionResult> Index(string movieGenre, string searchString)
        {
            // Use LINQ to get list of genres.
            IQueryable<string> genreQuery = from m in _context.Movie
                                            orderby m.Genre
                                            select m.Genre;
            var movies = from m in _context.Movie
                         select m;

            if (!string.IsNullOrEmpty(searchString))
            {
                movies = movies.Where(s => s.Title!.Contains(searchString));
            }

            if (!string.IsNullOrEmpty(movieGenre))
            {
                movies = movies.Where(x => x.Genre == movieGenre);
            }

            var movieGenreVM = new MovieGenreViewModel
            {
                Genres = new SelectList(await genreQuery.Distinct().ToListAsync()),
                Movies = await movies.ToListAsync()
            };

            return View(movieGenreVM);
        }
```

Thêm Tìm kiếm theo thể loại

- Cập nhật Index.cshtml trong Views/Movies/

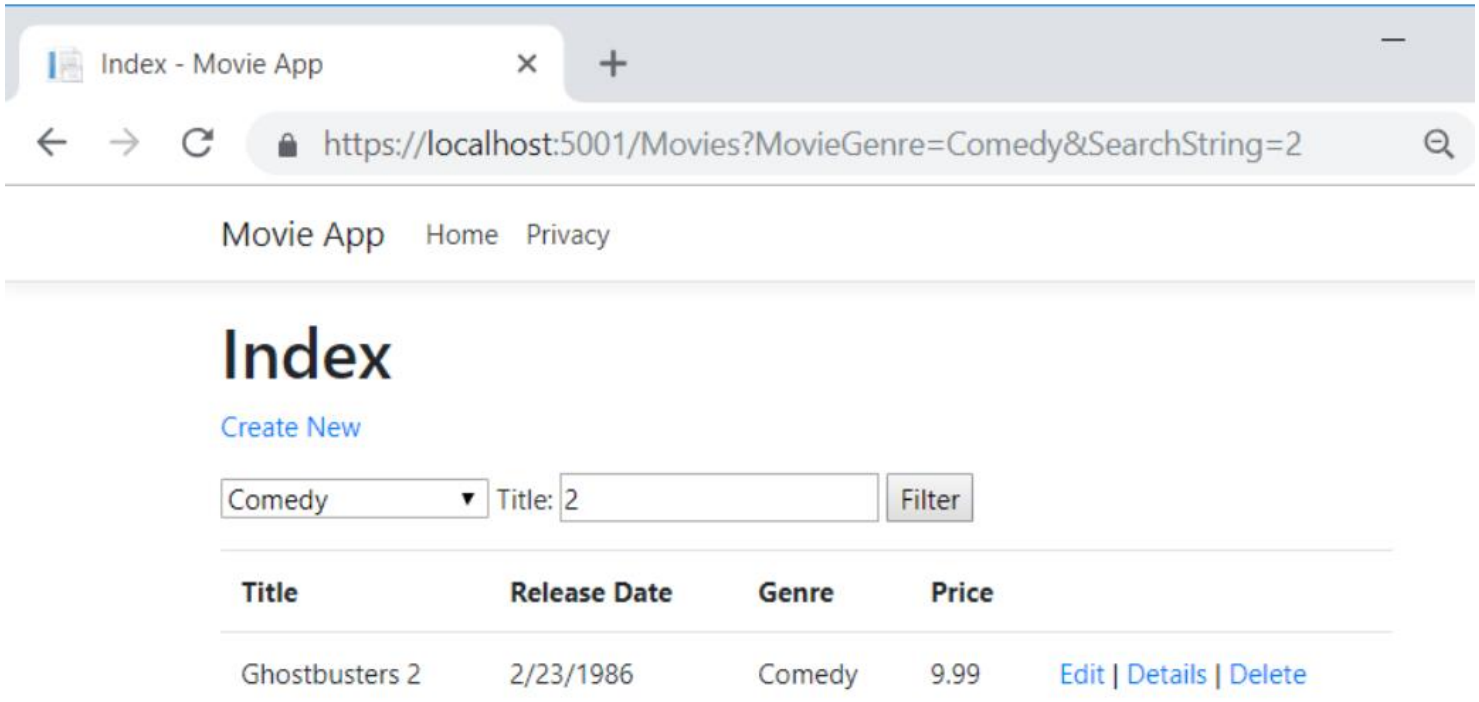
```
@model MvcMovie.Models.MovieGenreViewModel

<select asp-for="MovieGenre" asp-items="Model.Genres">
    <option value="">All</option>
</select>

Title: <input type="text" name="SearchString" />

<th>
    @Html.DisplayNameFor(model => model.Movies[0].Title)
</th>
...

@foreach (var item in Model.Movies)
```

Thêm Tìm kiếm theo thể loại

- Code Index.cshtml trong Views/Movies/ đầy đủ

```
@model MvcMovie.Models.MovieGenreViewModel

@{
    ViewData["Title"] = "Index";
}

<h1>Index</h1>

<p>
    <a asp-action="Create">Create New</a>
</p>
<form asp-controller="Movies" asp-action="Index" method="get">
    <p>
        <select asp-for="MovieGenre" asp-items="Model.Genres">
            <option value="">All</option>
        </select>

        Title: <input type="text" name="SearchString" />
        <input type="submit" value="Filter" />
    </p>
</form>

<table class="table">
    <thead>
        <tr>
            <th>
                @Html.DisplayNameFor(model => model.Movies[0].Title)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Movies[0].ReleaseDate)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Movies[0].Genre)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Movies[0].Price)
            </th>
            <th></th>
        </tr>
    </thead>
    <tbody>
@foreach (var item in Model.Movies) {
        <tr>
            <td>
                @Html.DisplayFor(modelItem => item.Title)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.ReleaseDate)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.Genre)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.Price)
            </td>
            <td>
                <a asp-action="Edit" asp-route-id="@item.Id">Edit</a> |
                <a asp-action="Details" asp-route-id="@item.Id">Details</a> |
                <a asp-action="Delete" asp-route-id="@item.Id">Delete</a>
            </td>
        </tr>
}
    </tbody>
</table>
```

Kết quả


Index - Movie App
https://localhost:5001/Movies?MovieGenre=Comedy&SearchString=2
Movie App
Home
Privacy
Index
Create New
Comedy
Title: 2
Filter
Title
Release Date
Genre
Price
Ghostbusters 2
2/23/1986
Comedy
9.99
Edit | Details | Delete

Thêm thuộc tính mới

Add a new field

- Entity Framework Code First Migrations được sử dụng để:
  - Thêm một trường mới vào mô hình.
  - Di chuyển trường mới vào cơ sở dữ liệu.
- EF Code First được sử dụng để tự động tạo cơ sở dữ liệu

Thực hiện

- Thêm một thuộc tính Rating vào Models/Movie.cs:

```
public decimal Price { get; set; }
public string? Rating {  get; set; }
```

- Cập nhật thuộc tính [Bind] cho method Create và Edit trong MoviesController.cs:

```
Create([Bind("Id,Title,ReleaseDate,Genre,Price,Rating")] Movie movie)
```

- Chỉnh sửa /Views/Movies/Index.cshtml và thêm Rating

```
@Html.DisplayNameFor(model => model.Movies[0].Rating)
```

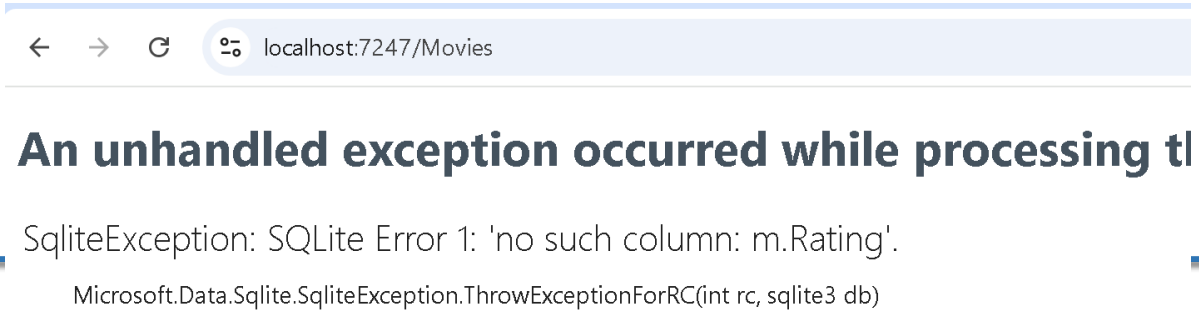
```
@Html.DisplayFor(modelItem => item.Rating)
```

Thực hiện

- Cập nhật lớp SeedData để cung cấp giá trị cho cột mới.

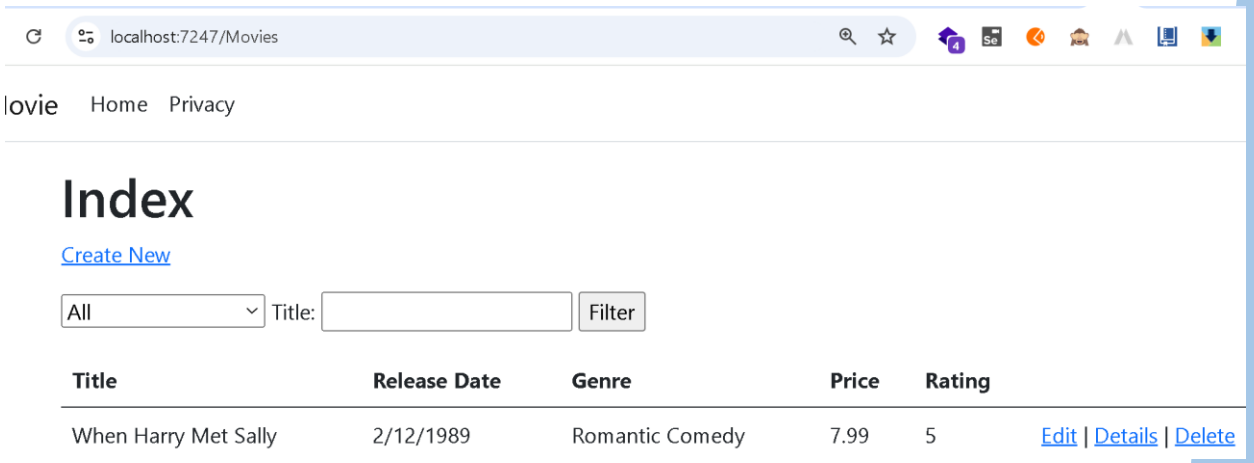
```
new Movie
    {
        Title = "When Harry Met Sally",
        ReleaseDate = DateTime.Parse("1989-2-12"),
        Genre = "Romantic Comedy",
        Rating = "R",
        Price = 7.99M
    }
```

- F5 chạy chương trinh



Thực hiện

- Sử dụng Code First Migrations để cập nhật lược đồ cơ sở dữ liệu
  - dotnet ef migrations add Rating
  - dotnet ef database update
- F5 chạy lại



Tạo những ràng buộc nhập liệu

Thêm Validation

- Phần này hướng dẫn thêm các Validation để kiểm tra dữ liệu từ người dùng nhập liệu vào.

- Mục tiêu là **viết 1 lần, sửa chữa 1 chỗ và sử dụng nhiều chỗ.**

- **Keeping Things DRY ("Don't Repeat Yourself")**

Thêm Validation

- Thêm namespace **System.ComponentModel.DataAnnotation** vào các lớp code để định nghĩa cho các thuộc tính
- Các thuộc tính Validation:
  - Required
  - StringLength
  - RegularExpression
  - Range

Add validation rules to the movie model

- Cập nhật lớp Movie để dùng các thuộc tính xác thực Required, StringLength, RegularExpression, và Range tích hợp sẵn

```
using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace MvcMovie.Models
{
    public class Movie
    {
        public int Id { get; set; }

        [StringLength(60, MinimumLength = 3)]
        [Required]
        public string? Title { get; set; }

        [Display(Name = "Release Date")]
        [DataType(DataType.Date)]
        public DateTime ReleaseDate { get; set; }

        [Range(1, 100)]
        [DataType(DataType.Currency)]
        [Column(TypeName = "decimal(18, 2)")]
        public decimal Price { get; set; }

        [RegularExpression(@"^[A-Z]+[a-zA-Z\s]*$")]
        [Required]
        [StringLength(30)]
        public string? Genre { get; set; }

        [RegularExpression(@"^[A-Z]+[a-zA-Z0-9""'\s-]*$")]
        [StringLength(5)]
        [Required]
        public string? Rating { get; set; }
    }
}
```

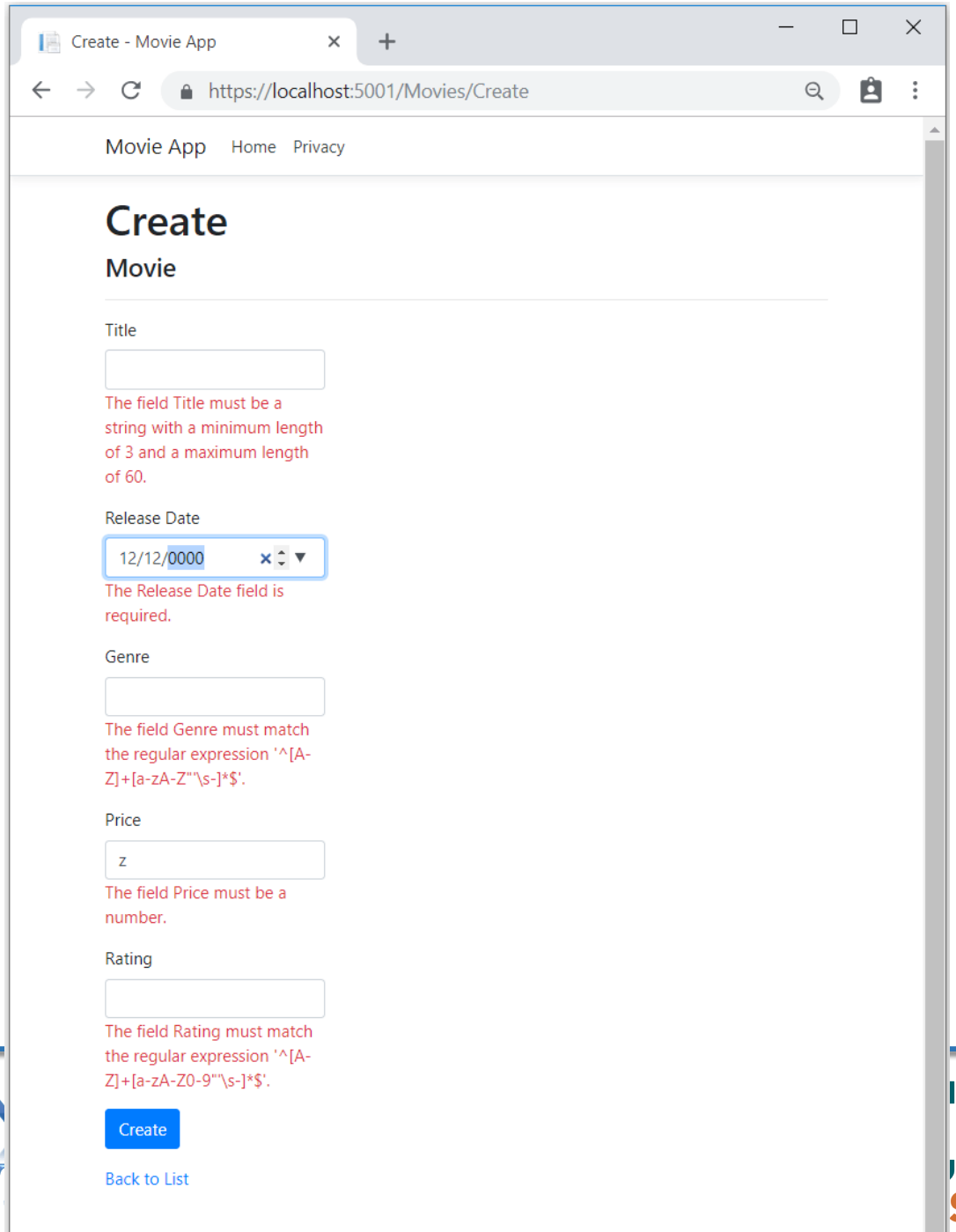
Giải thích

- `[RegularExpression(@"^[A-Z]+[a-zA-Z\s]*$")]`
  - Chỉ được sử dụng chữ cái.
  - Chữ cái đầu tiên phải viết hoa. Khoảng trắng được phép nhưng không được phép có số và ký tự đặc biệt.
- [RegularExpression(@"^[A-Z]+[a-zA-Z0-9"""\s-]*$")]
  - Yêu cầu ký tự đầu tiên phải là chữ in hoa.
  - Cho phép các ký tự đặc biệt và số ở các khoảng trống tiếp theo.
- Các kiểu giá trị (như decimal, int, float, DateTime) về cơ bản là bắt buộc và không cần [Required]thuộc tính

Thêm Validation

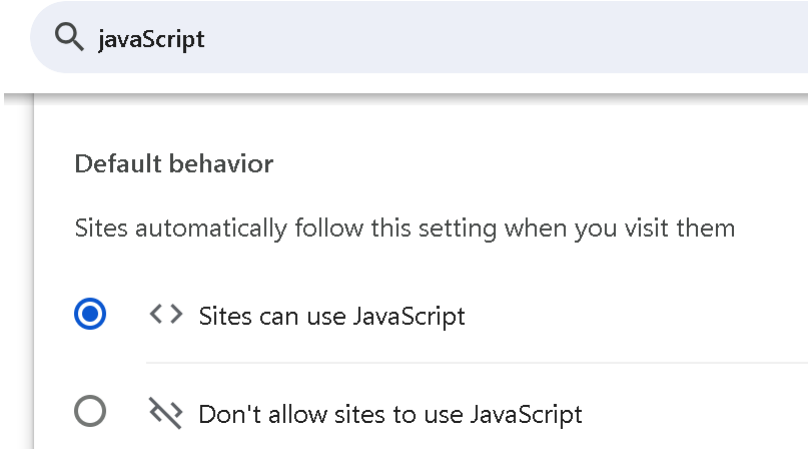
- Thuộc tính StringLength sẽ thiết lập độ dài chuỗi trong database.
- Nếu dùng Code First thì khi update mô hình code database sẽ thiết lập độ dài các trường nvarchar theo độ dài đã thiết lập => **DB đã thay đổi**
- **Sử dụng Code First Migrations để cập nhật lược đồ cơ sở dữ liệu**
  - **dotnet ef migrations add ChangeMovie**
  - **dotnet ef database update**

Thực hiện


Create - Movie App
https://localhost:5001/Movies/Create
Movie App
Home
Privacy
Create
Movie
Title
The field Title must be a string with a minimum length of 3 and a maximum length of 60.
Release Date
12/12/0000
The Release Date field is required.
Genre
The field Genre must match the regular expression '^[A-Z]+[a-zA-Z''\s-]*$'.
Price
z
The field Price must be a number.
Rating
The field Rating must match the regular expression '^[A-Z]+[a-zA-Z0-9''\s-]*$'.
Create
Back to List

Kiểm tra xác thực cả client và server

- Tắt js trong Web Browser (ví dụ Chrome)



- Hoạt dùng các công cụ để lắng nghe sự kiện như [fiddler]

Thêm Validation

- Nếu disable **Javascript** ở trình duyệt thì sẽ không thực hiện được Validation.

- Trường hợp Javascript bị disable ở client thì **phải sử dụng bắt lỗi và trả kết quả lỗi ở server.**

```
public async Task<IActionResult> Create([
{
    if (ModelState.IsValid)
    {
```

Thêm Validation

- Nếu sử dụng **Database first** Để tránh mất mát các định nghĩa Validation cho các thuộc tính khi cập nhập mô hình Code thì nên viết các lớp Metadata riêng để định nghĩa.
- Xem thêm tại: https://docs.microsoft.com/en-us/aspnet/mvc/overview/getting-started/database-first-development/enhancing-data-validation

Sử dụng Thuộc tính DataType

- Lợi ích của Thuộc tính DataType
  - Trình duyệt có thể bật các tính năng HTML5 (ví dụ: để hiển thị điều khiển lịch, ký hiệu tiền tệ phù hợp với địa phương, liên kết email, v.v.)
  - Thuộc tính này DataTypecó thể cho phép MVC chọn đúng mẫu trường để hiển thị dữ liệu

Sử dụng Thuộc tính DataType

```
using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace MvcMovie.Models
{
    public class Movie
    {
        public int Id { get; set; }

        [StringLength(60, MinimumLength = 3)]
        public string Title { get; set; }

        [Display(Name = "Release Date"), DataType(DataType.Date)]
        public DateTime ReleaseDate { get; set; }

        [RegularExpression(@"^[A-Z]+[a-zA-Z\s]*$"), Required, StringLength(30)]
        public string Genre { get; set; }

        [Range(1, 100), DataType(DataType.Currency)]
        [Column(TypeName = "decimal(18, 2)")]
        public decimal Price { get; set; }

        [RegularExpression(@"^[A-Z]+[a-zA-Z0-9""'\s-]*$"), StringLength(5)]
        public string Rating { get; set; }
    }
}
```

Bài tập

- Việt hoá Movie class
- Thêm 1 column Tóm tắc
- Xác lập các ràng buộc:
  - Title: số lượng ký tự từ 3-200
  - Tóm tắc: trên 10 ký tự
  - Release Date: lớn hơn ngày hiện tại, hiện thị kiểu ngày tháng năm
  - Genre: chỉ cho phép nhập từ danh sách có sẳn: Kiếm Hiệp, XHĐ, Thiếu nhi...
  - Price: lớn hơn 5000
  - Rated: chọn từ 1-5

UEH UNIVERSITY

Q&A